

TD n°5 : Initiation à l’algorithmique

Derrez Lorenzo

October 6, 2026

Exercice 0

Exercice 1

Question 1:

On obtient les tableaux suivant :

- $T = [10, 20, 60, 50, 30, 40]$
- $T = [10, 20, 60, 50, 30, 40]$
- $T = [10, 20, 30, 50, 60, 40]$
- $T = [10, 20, 30, 40, 60, 50]$
- $T = [10, 20, 30, 40, 50, 60]$
- $T = [10, 20, 30, 40, 50, 60]$

Question 2

Jeu de 6 exemples pour *indiceDuMinimum* :

- $\left\{ \begin{array}{l} E : T = [1], i = 0 \\ S : j = 0 \end{array} \right.$
- $\left\{ \begin{array}{l} E : T = [1, 1], i = 0 \\ S : j = 0 \end{array} \right.$
- $\left\{ \begin{array}{l} E : T = [3, 6, 2], i = 0 \\ S : j = 2 \end{array} \right.$
- $\left\{ \begin{array}{l} E : T = [1, 6, 7, 2], i = 1 \\ S : j = 3 \end{array} \right.$
- $\left\{ \begin{array}{l} E : T = [1, 6, 2, 2], i = 1 \\ S : j = 2 \end{array} \right.$
- $\left\{ \begin{array}{l} E : T = [12, 3, 10], i = 2 \\ S : j = 2 \end{array} \right.$

Question 3

On définit le problème *IndiceDuMinimum* :

Algorithm 1 *IndiceDuMinimum*

```
1: E : un tableau  $T$  d'entiers, et un entier  $i$  compris entre 0 et  $|T| - 1$ 
2: S : un entier  $j$  compris entre  $i$  et  $|T| - 1$  tel que  $T[j]$  est le minimum entre  $i$  et  $|T|-1$ .
3: def IndiceDuMinimum( $i$ :int,  $T$ :Array[integer]) -> int:
4:     int  $n \leftarrow \text{size}(T)$ 
5:     int  $\text{min\_act} \leftarrow T[i]$ 
6:     int  $\text{ind\_act} \leftarrow i$ 
7:     for  $k$  in range( $i, n$ ):
8:         if  $T[k] < \text{min\_act}$ :
9:              $\text{min\_act} \leftarrow T[k]$ 
10:             $\text{ind\_act} \leftarrow k$ 
11: return  $\text{ind\_act}$ 
```

Evaluation de la complexité en espace

- On ne crée que des variables entières à chaque appel
- $\theta(1)$, et c'est optimal

Evaluation de la complexité temporelle

- création des variables entières : $\theta(1)$
- Parcours de i à $|T|-1$: $\theta(|T| - 1 - i)$
- Pour $n = |T|-1-i$ la taille de l'entrée : $\theta(n)$
- Etant donné que la comparaison et les affectations sont en temps constant
- La complexité **totale** est $\theta(n)$, et c'est optimal (car theta).

Question 4

Invariant de boucle pour cet algorithme : "*Tout élément de $[i, |T|]$ est inférieur ou égal à min_act .*"

Exercice 2**Question 1**

Listing 1: sontCompatibles

```
1
2 def sontCompatibles( $i$ :int,  $j$ :int, datedebut:Array[int], datefin:Array[int]) -> bool :
3     int  $\text{fin\_i} = \text{datefin}[i]$ 
4     int  $\text{debut\_j} = \text{datedebut}[j]$ 
5     # Par symetrie des roles de  $i$  et  $j$  on peut supposer que  $i$  est le premier evenement (
6     entre autre  $i < j$ )
7     if  $\text{fin\_i} < \text{debut\_j}$  :
8         return true
9     else:
10        return false
```

Question 2

Supposons que l'on connaisse un évènement sélectionné.

Dès lors l'ensemble des évènements incompatibles avec ce dernier sont incompatibles pour l'ensemble sélectionné.

Question 3

Hypothèse : Choisir en 1^{er} élément un évènement de durée minimale.

Ce n'est pas forcément optimal.

Contre exemple :

En reprenant l'exemple du cours, si l'on suit cette méthode on peut obtenir {5, 6, 7, 8}

Or il existe au moins un sous ensemble de cardinal supérieur : { 1, 3, 4, 6, 7}

Question 4

Hypothèse : Choisir comme 1^{er} élément un évènement de date de fin minimale.

Cette hypothèse semblerait contribuer à une solution optimale.

Exemple :

En reprenant l'exemple du cours, on peut obtenir en appliquant cette hypothèse le sous-ensemble de cardinalité maximale : {3, 4, 5, 6, 7}

Question 5

Remarque : Pour cette première méthode, on utilise la fonction de l'exercice 1 *IndiceDuMinimum* en ajoutant une comparaison ($\theta(1)$ *encomplexité temporelle*) qui permet de vérifier que le minimum n'a pas déjà été traité avant. Donc on garde une complexité temporelle de $\theta(n)$ pour cette fonction.

Algorithm 2 Première méthode

```
1: E : i, j deux entiers, datedebut et datefin deux tableaux d'entiers
2: S : un tableau Res de booléen
3: def Selection_evt(i:int, j:int, datedebut:int, datefin:int) -> Array[bool] :
4:   int n ← size(datedebut)
5:   Array[bool] selectionne ← New_Array(n, false)
6:   Array[bool] visites ← New_Array(n, false)
7:   int marquage ← 0
8:   While marquage < n:
9:     int j ← indiceDuMinimum(fin, visites)
10:    selectionne[j] ← true
11:    visites[j] ← true
12:    marquage ← marquage + 1
13:    for k in range(n):
14:      if not visites[k]:
15:        if not sontCompatibles(k, j, datedebut, datefin):
16:          marquage ← marquage + 1
17:          visites[k] ← true
18:    Libérer la mémoire de visites en fin de programme
19: return selectionne
```

Evaluation de la complexité en espace

- 2 tableaux initialisés avec n la taille de l'entrée
- Un des deux tableaux est libéré à la fin de l'exécution
- $\theta(2n) \implies \theta(n)$

Evaluation de la complexité temporelle

- chaque tour coûte $\theta(n)$
- Il peut y avoir n tours de boucles
- **Pire cas :** $\theta(n^2)$

Question 6

Il existe une méthode permettant d'améliorer la complexité temporelle de l'algorithme précédent :

- effectuer un tri adapté en $\theta(n \log(n))$ selon la fin minimale j .
- Parcourir une unique fois en $\theta(n)$.
- On sélectionne chaque évènement compatible avec le dernier sélectionné.

Ainsi on aurait en temps, puis en espace :

- **Complexité temporelle :** $\#f(n) = \theta(n \log(n)) + \theta(n) = \theta(n \log(n))$
- **Complexité en espace :** $\theta(n)$ (inchangé par rapport à Q5)

Récriture de l'algorithme :